

Project Submission Description: Community Hero

This document is the end-to-end project description for Vibe2Ship's Civic AI Co-Pilot application. It explains the problem, the product solution, user roles, architecture, technical implementation, and the complete set of included features.

1. Project Vision

Community Hero transforms local infrastructure reporting into a trusted, AI-supported civic workflow.

The platform is designed for two primary audiences:

- **Citizens** who want to report issues quickly and see follow-up progress.
- **Field workers and municipal teams** who need verified evidence, better dispatch, and AI audit confidence.

This app combines modern web UI, AI analysis, and a scalable backend to make civic issue resolution more transparent and effective.

2. Problem Statement

Most civic reporting systems suffer from slow response, fragmented tools, and low trust.

Common issues in local infrastructure reporting include:

- reports scattered across different portals,
- little context for municipal departments,
- no community verification layer,
- few controls for proof of repair,
- unclear ownership and status tracking.

2.1 Challenge Scope

Community Hero focuses on hyperlocal infrastructure issues, including:

- potholes,
- streetlight failures,
- overflowing trash,
- storm drain blockages,
- sidewalk damage,
- water leaks.

2.2 Desired Outcome

The product aims to:

- simplify citizen reporting,
 - automate issue classification,
 - provide clear role-based workflows,
 - enable community validation,
 - offer AI-backed repair verification,
 - and make the entire process auditable and accountable.
-

3. What the Solution Includes

Community Hero is built as a complete full-stack product with the following components:

3.1 Core Application Areas

- **Visual issue reporting** with image/video upload,
- **AI-assisted report generation** using Google Gemini,
- **Geo-location and interactive mapping** for exact issue placement,
- **Community verification** through upvotes and flags,
- **Field worker task management** with status progression,
- **AI repair proof audit** using evidence validation,
- **Gamification** via points and leaderboard,
- **Analytics dashboard** for live civic metrics.

3.2 Technical Stack

- Frontend: **React** with **Vite**, styled for a modern dark theme.
 - Backend: **FastAPI** with modular router and repository layers.
 - Database: **MongoDB** as the primary store, with **SQLite** fallback for local dev.
 - AI: **Google Gemini** for multimodal classification and audit review.
-

4. Detailed Feature Breakdown

4.1 Visual Issue Reporting

Citizens can submit a new hazard report with only an uploaded photo or video.

The report flow includes:

- image/video capture,
- automatic AI-considered title and description,
- category and severity inference,
- department routing suggestions,
- priority and estimated completion guidance,
- mapping the issue to an exact location.

4.2 AI-Powered Pre-Fill and Classification

Google Gemini analyzes visual evidence and returns structured metadata:

- issue category,
- severity score,
- concise title,
- descriptive summary,
- recommended department,
- dispatch directive,
- internal priority score.

This dramatically reduces the amount of manual form input required from the user.

4.3 Geo-Location and Mapping

The frontend integrates a map-based coordinate selector.

Key map features include:

- default geotag from the upload flow,
- draggable pin placement,
- visible issue markers,
- location summaries on each ticket card.

4.4 Community Verification and Moderation

Citizen verification is built into the issue lifecycle.

Users can:

- upvote issues to confirm legitimacy,
- flag duplicates or incorrect submissions,
- view verification history and trust scores.

This layer improves data quality and prevents unnecessary work.

4.5 Role-Based Workflow

The platform supports two main roles:

- **Citizen**: reports hazards, verifies issues, and tracks outcomes.
- **Field Worker**: views assigned tasks, advances issue status, and submits proof of repair.

The UI adapts to each role with role-specific actions and task lists.

4.6 AI Resolution Proof Audit

When a worker submits a repair proof image, the backend uses Gemini to verify the evidence.

The audit process includes:

- comparing proof imagery to the original issue,
- generating verification feedback,
- updating the ticket to **Resolved** if proof is confirmed,
- or returning the issue to **In Progress** if the proof is insufficient.

This creates an AI-backed certification layer for completed work.

4.7 Dashboard and Insights

The dashboard provides live metrics and civic insights:

- total active issues,
- resolved issue count,
- high-priority hazards,
- community contribution points,
- team assignments and backlog status.

It also surfaces predictive risk signals based on AI analysis.

4.8 Gamification and Engagement

Community Hero rewards participation with:

- points for reporting issues,
- points for verifying and flagging issues,
- a public leaderboard for top contributors,
- badges for sustained engagement.

This encourages sustained civic involvement.

5. Architecture and Implementation

5.1 Frontend Architecture

The frontend is a React application built with Vite.

Important modules include:

- ``frontend/src/components/Dashboard.jsx`` — the main issue and analytics UI,
- ``frontend/src/components/ReportIssue.jsx`` — issue creation, AI prefill, and map editing,
- ``frontend/src/components/IssueMap.jsx`` — geographic issue visualization,

- ``frontend/src/components/Leaderboard.jsx`` — community scoring and ranking,
- ``frontend/src/context/IssueContext.jsx`` — global state management and backend communication.

The frontend connects to the backend using the ``/api`` proxy routes configured in ``vite.config.js``.

5.2 Backend Architecture

FastAPI hosts the backend with clear separation of concerns:

- ``backend/main.py`` boots the app and mounts routers,
- ``backend/routers/issues.py`` handles issue lifecycle operations,
- ``backend/routers/analysis.py`` manages media uploads and AI classification,
- ``backend/routers/users.py`` provides leaderboard and profile endpoints.

The backend also includes:

- ``backend/models/schemas.py`` for request/response validation,
- ``backend/repositories/issue_repository.py`` for database operations,
- ``backend/repositories/user_repository.py`` for score and profile handling,
- ``backend/services/gemini_service.py`` for AI prompt creation and response parsing.

5.3 Database Design

Primary persistence is handled by MongoDB using collections for:

- ``issues``,
- ``users``,
- ``activity`` or ``audit logs``.

A local SQLite fallback is supported, allowing the project to run without a MongoDB instance in development.

5.4 AI Integration Strategy

The AI service is integrated via Gemini with the following patterns:

- multimodal evidence classification for new reports,
- structured output parsing for data consistency,
- repair proof verification and feedback generation,
- fallback handling for incomplete AI responses.

5.5 Environment and Configuration

The project uses environment variables for secure configuration:

- ``VITE_GEMINI_API_KEY`` for frontend/AI access,
- ``MONGODB_URI`` and ``MONGODB_DB_NAME`` for the backend database,
- optional local fallback behavior when MongoDB is unavailable.

6. Included Files and Components

This project includes the following deliverables:

Root-Level Files

- `.env.example` — environment variable template,
- `package.json` — root scripts and dependency automation,
- `README.md` — general project documentation,
- `PROJECT_DESCRIPTION.md` and `PROJECT_DESCRIPTION.pdf` — detailed submission documentation,
- `scripts/md_to_pdf.py` — PDF generator for the markdown description.

Backend Files

- `backend/config.py` — environment loading and database mode selection,
- `backend/main.py` — FastAPI application entrypoint,
- `backend/database/connection.py` — MongoDB/SQLite connection management,
- `backend/database/setup.py` — collection initialization and seeded demo data,
- `backend/models/schemas.py` — Pydantic models and validation,
- `backend/repositories/issue_repository.py` — CRUD and lifecycle methods,
- `backend/repositories/user_repository.py` — leaderboard and point logic,
- `backend/routers/analysis.py` — media upload AI analysis endpoint,
- `backend/routers/issues.py` — issue management endpoints,
- `backend/routers/users.py` — user/leaderboard endpoints,
- `backend/services/gemini_service.py` — Gemini prompt and response service.

Frontend Files

- `frontend/index.html` — app shell,
- `frontend/package.json` — frontend dependencies,
- `frontend/vite.config.js` — development proxy config,
- `frontend/src/App.jsx` — core app structure,
- `frontend/src/App.css` — theme and layout styling,
- `frontend/src/index.css` — base global styles,
- `frontend/src/main.jsx` — React application bootstrap.

7. Running the Project

Local Development

1. Install all dependencies:

```
```bash
npm run install:all
```
```

2. Create or update `.env` from `.env.example`.

3. Start the app:

```
```bash
npm run dev
```
```

Backend Startup

Start the backend from the project root using:

```
```bash
python -m uvicorn backend.main:app --host 127.0.0.1 --port 8000 --log-level info
```
```

API Documentation

The backend exposes OpenAPI docs at:

```
```text
http://127.0.0.1:8000/docs
```
```

8. Future Enhancements

Potential next steps for the project include:

- adding user authentication and role-based access control,
- storing and serving media files in cloud storage,
- improving AI prompt robustness with feedback loops,
- adding an admin panel for department dashboards,
- enabling real-time notifications for issue updates.

9. Why This Solution Matters

Community Hero is designed to make civic issue reporting:

- faster to submit,
- easier to verify,
- more accountable,

- more useful for municipal response teams,
- more engaging for residents.

It also demonstrates a practical AI-enabled civic workflow with real-world applications for local infrastructure and community resilience.

- community engagement through gamified points and badges.

10. Future Enhancements

Potential next steps include:

- real-time WebSocket updates for immediate issue status changes,
- mobile-friendly upload flow with camera capture,
- advanced duplicate detection using spatial clustering,
- richer AI proof validation via object detection,
- multilingual citizen reporting,
- exportable municipal performance dashboards.

11. Repo Structure Summary

```
```text
Vibe2Ship/
├── backend/
│ ├── database/
│ ├── models/
│ ├── repositories/
│ ├── routers/
│ ├── services/
│ ├── static/
│ ├── main.py
│ └── requirements.txt
├── frontend/
│ ├── src/
│ │ ├── components/
│ │ ├── context/
│ │ └── utils/
│ ├── package.json
│ └── vite.config.js
├── .env.example
└── package.json
```



■■■ README.md  
■■■ PROJECT\_DESCRIPTION.md  
^^^

---

## 12. Conclusion

---

Community Hero is a complete end-to-end civic AI co-pilot. It guides citizens from hazard detection to municipal verification and resolution, while giving municipal workers a practical workflow to claim, execute, and certify repairs. The project demonstrates a strong fusion of AI, geospatial mapping, verification workflows, and community-driven civic engagement.